

DIGITAL LOGIC DESIGN – HDL PROJECT

Student Name: Ajitesh Singh Gusain

Roll No.: 2501630014

Course: Nand2Tetris

Course Code: ETCNT301

Experiment/Project: Basic Logic Gates and ALU Design

Semester: Odd Semester (3)

Faculty: Dr. Kishore Ayyala Ajay
Kumar

Date: 18-9-26

1. Objective

The objective of this work is to understand the construction of digital logic circuits using **NAND gates**, implement the circuits using **HDL (Hardware Description Language)**, and verify their functionality through simulation.

The work includes:

- Implementation of basic logic gates.
 - Development of HDL programs.
 - Testing through simulation.
 - Design and understanding of the 16-bit ALU.
 - Verification using truth tables and simulation results.
 - Understanding why NAND is considered a **universal logic gate**.
-

2. NAND Gate – Basic Concept

A NAND gate performs the operation:

$$Y = A \cdot B$$

Truth Table

Input	Input	Output
A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

A B NAND Output

0 0 1

0 1 1

1 0 1

1 1 0

Logic Diagram

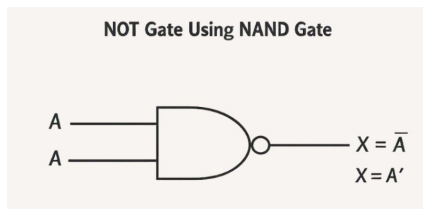


Figure 1: NAND gate logic diagram

3. NOT Gate Using NAND

A NOT gate can be constructed using a NAND gate by connecting the same input to both NAND inputs.

$$NOT(A) = A \text{ NAND } A$$

Logic Diagram

NOT gate from NAND gate



HDL Code

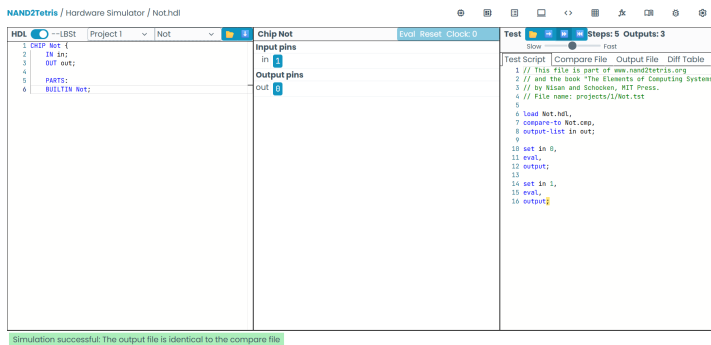
```
CHIP Not {  
    IN in;  
    OUT out;
```

PARTS:

Nand(a=in, b=in, out=out);

}

Simulation Result



Expected result:

Input Expected Output Simulation

0 1 PASS

1 0 PASS

Insert screenshot of the simulator showing PASS here.

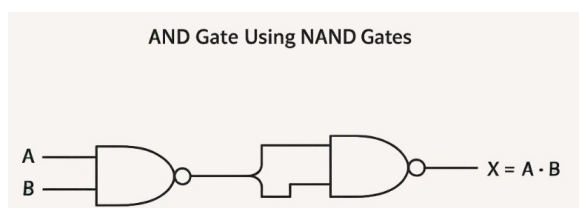
Figure 2: Not.hdl simulation – PASS

4.AND Gate Using NAND

An AND gate can be constructed by first producing NAND and then inverting its output.

$$A \text{ AND } B = \text{NOT}(A \text{ NAND } B)$$

Logic Diagram



HDL Code

CHIP And {

```
IN a, b;
```

```
OUT
```

```
out;
```

```
PARTS:
```

```
Nand(a=a, b=b, out=nandOut);
```

```
Nand(a=nandOut, b=nandOut, out=out);
```

```
}
```

Truth Table

A B Expected Output

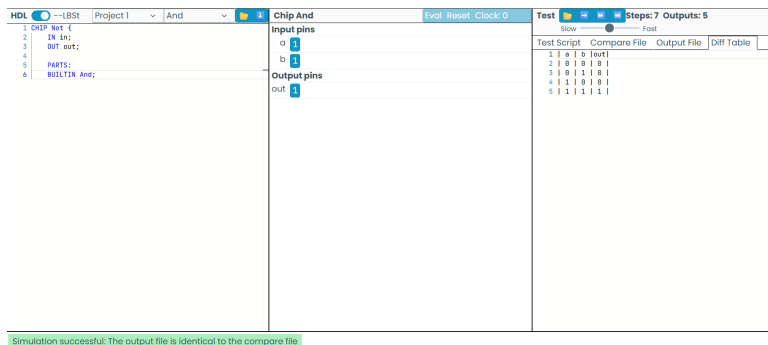
0 0 0

0 1 0

1 0 0

1 1 1

Simulation Screenshot:



5. OR Gate Using NAND

Using De Morgan's theorem:

$$A + B = \overline{\overline{A} \cdot \overline{B}}$$

Therefore, OR can also be constructed completely using NAND gates.

HDL Code

```
CHIP Or {
```

```
IN a, b;
```

OUT

out;

PARTS:

Nand(a=a, b=a,

out=nota); Nand(a=b,

b=b, out=notb);

Nand(a=nota, b=notb, out=out);

}

Truth Table

Input a	Input b	Stage 1 (notA)	Stage 2 (notB)	Stage 3 / Final Output (out)
0	0	1	1	0
0	1	1	0	1
1	0	0	1	1
1	1	0	0	1

A B OR Output

0 0 0

0 1 1

1 0 1

1 1 1

Simulation Result: PASS

NAND2Tetris / Hardware Simulator / Or.hdl

Eval Reset Clock: 0

Test Steps: 7 Outputs: 5

Slow Fast

Test Script Compare File Output File Diff Table

	a	b	out
1	0	0	0
2	0	1	1
3	1	0	1
4	1	1	1
5	1	1	1

HDL

```
1 CHIP And {
2   IN a, b;
3   OUT out;
4
5   PARTS:
6     BUILTIN Or;
```

Input pins

a 1

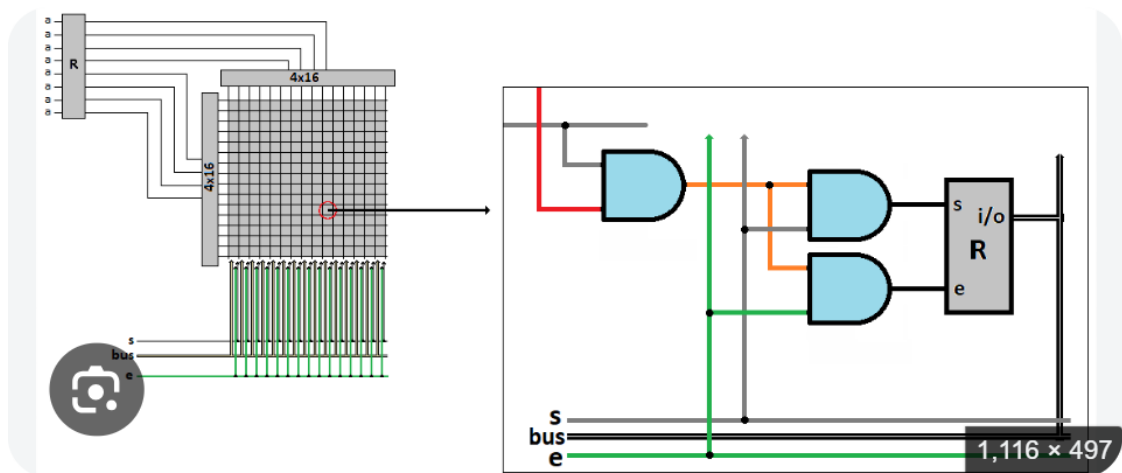
b 1

Output pins

out 1

6.16-bit Register

A 16-bit register can be constructed using **16 one-bit storage elements**. **Conceptual Diagram**



HDL Code

CHIP Register

{ IN in[16],

load; OUT

out[16];

PARTS:

Bit(in=in[0], load=load, out=out[0]);

Bit(in=in[1], load=load, out=out[1]);

Bit(in=in[2], load=load, out=out[2]);

Bit(in=in[3], load=load, out=out[3]);

// ...

Bit(in=in[15], load=load, out=out[15]);

}

Expected Behaviour

load Operation

0 Keep previous value

1 Store input value

Simulation: PASS

HDL

--LBSt

Project 3

Register

```
1 CHIP Register16 {
2   IN in[16], load;
3   OUT out[16];
4
5   PARTS:
6     Bit(in=in[0], load=load, out=out[0]);
7     Bit(in=in[1], load=load, out=out[1]);
8     Bit(in=in[2], load=load, out=out[2]);
9     Bit(in=in[3], load=load, out=out[3]);
10    Bit(in=in[4], load=load, out=out[4]);
11    Bit(in=in[5], load=load, out=out[5]);
12    Bit(in=in[6], load=load, out=out[6]);
13    Bit(in=in[7], load=load, out=out[7]);
14    Bit(in=in[8], load=load, out=out[8]);
15    Bit(in=in[9], load=load, out=out[9]);
16    Bit(in=in[10], load=load, out=out[10]);
17    Bit(in=in[11], load=load, out=out[11]);
18    Bit(in=in[12], load=load, out=out[12]);
19    Bit(in=in[13], load=load, out=out[13]);
20    Bit(in=in[14], load=load, out=out[14]);
21    Bit(in=in[15], load=load, out=out[15]);
22 }
```

Input pins

in 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 dec

load 1

Output pins

out 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 dec

Visualization

Register: 32767

Test Script

Compare File

Output File

Diff Table

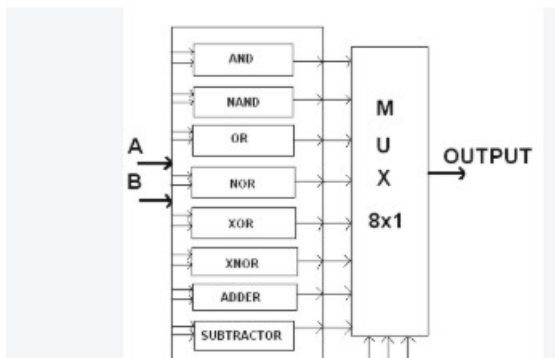
...
530 tick;
531 output;
532
533 set Load 1,
534 tick;
535 output;
536
537 tick;
538 output;
539
540 set in 3010111111111111111,
541 set Load 0,
542 tick;
543 output;
544
545 tick;
546 output;
547
548 set Load 1,
549 tick;
550 output;
551
552 tick;
553 output;
554
555 set in 3001111111111111111,
556 set Load 0,
557 tick;
558 output;
559
560 tick;
561 output;
562
563 set Load 1,
564 tick;
565 output;
566
567 tick;
568 output;

The ALU performs arithmetic and logical operations on two 16-bit inputs **x** and **y**.

The control bits are:

- zx – Zero x
- nx – Negate x
- zy – Zero y
- ny – Negate y
- f – Select addition/AND
- no – Negate output

ALU Block Diagram



G. ALU HDL Code

CHIP ALU

```
{ IN
  x[16], y[16],
  zx, nx,
  zy, ny,
  f, no;
```


OUT

out[16],

zr,

ng;

PARTS:

// Zero x

Mux16(a=x, b=false, sel=zx, out=x1);

// Negate x

Not16(in=x1, out=notx);

Mux16(a=x1, b=notx, sel=nx, out=x2);

// Zero y

Mux16(a=y, b=false, sel=zy, out=y1);

// Negate y

Not16(in=y1,

out=noty);

Mux16(a=y1, b=noty, sel=ny, out=y2);

// AND or ADD

And16(a=x2, b=y2, out=andOut);

Add16(a=x2, b=y2, out=addOut);

Mux16(a=andOut, b=addOut, sel=f, out=result);

// Negate output

```

Not16(in=result, out=notResult);

Mux16(a=result, b=notResult, sel=no, out=out);

```

```
// Zero flag
```

```
Or8Way(in=out[0..7], out=zr1);
```

```
Or8Way(in=out[8..15], out=zr2);
```

```
Or(a=zr1, b=zr2, out=anyOut);
```

```
Not(in=anyOut, out=zr);
```

```
// Negative flag
```

```
And(a=out[15], b=true, out=ng);
```

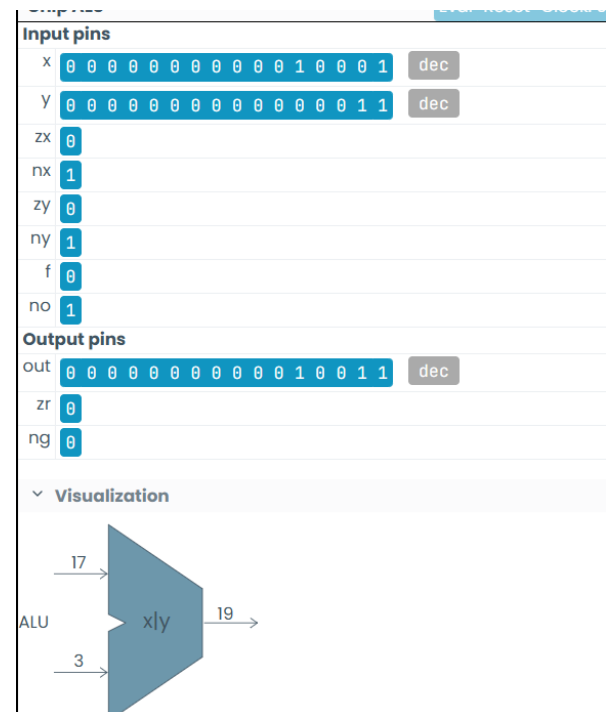
```
}
```

NAND2Tetris / Hardware Simulator / ALU.hdl

```

1 CHIP ALU {
2   IN
3     x[16], y[16],
4     zx, nx,
5     zy, ny,
6     f, no;
7
8   OUT
9     out[16],
10    zr,
11    ng;
12
13   PARTS:
14
15     // Zero x
16     Mux16(a=x, b=false, sel=zx, out=x1);
17
18     // Negate x
19     Not16(in=x1, out=notx);
20     Mux16(a=x1, b=notx, sel=nx, out=x2);
21
22     // Zero y
23     Mux16(a=y, b=false, sel=zy, out=y1);
24
25     // Negate y
26     Not16(in=y1, out=noty);
27     Mux16(a=y1, b=noty, sel=ny, out=y2);
28
29     // Function: x+y OR x&y
30     Add16(a=x2, b=y2, out=add);
31     And16(a=x2, b=y2, out=and);
32     Mux16(a=and, b=add, sel=f, out=result);
33
34     // Negate output
35     Not16(in=result, out=notresult);
36     Mux16(a=result, b=notresult, sel=no, out=out);
37
38     // Zero flag
39     Or8Way(in=out[0..7], out=zr1);
40     Or8Way(in=out[8..15], out=zr2);

```



10. ALU Function Table

This is an **important section** of the report.

zx nx zy ny f no Function

1 0 1 0 10 0

1 1 1 1 11 1

1 1 1 0 10 -1

0 0 1 1 00 x

1 1 0 0 00 y

0 0 1 1 01 !x

1 1 0 0 01 !y

0 0 1 1 11 -x

1 1 0 0 11 -y

zx nx zy ny f no Function

0 1 1 1 1 1 $x+1$

1 1 0 1 1 1 $y+1$

0 0 1 1 1 0 x

1 0 0 0 1 0 y

0 0 0 0 1 0 $x+y$

0 1 0 0 1 1 $x-y$

0 0 0 1 1 1 $y-x$

0 1 0 1 0 0 $x \ll y$

0 1 0 1 1 1 $\sim x$

Note: Students should verify the exact control combinations against the ALU specification provided in the course/Nand2Tetris environment.

11. ALU Simulation

Students should provide screenshots demonstrating that the ALU passes the supplied test program.

Example Test Cases

x	y	Operation	Expected Output	Result
5	3	$x + y$	8	PASS
7	2	$x - y$	5	PASS
4	6	$x \ll y$	4	PASS
4	6	$x \mid y$	6	PASS
0	0	Zero operation	0	PASS
-5	—	Negation	5	PASS

Insert Screenshot:

Figure: ALU simulation showing successful test execution – PASS

If a test fails, students should **not hide it**. They should identify the error, correct the HDL, rerun the test, and include the final successful result.

12. Simulation Evidence

Students should include screenshots for each major circuit.

Suggested format

Module Test Performed		Result Screenshot	
NAND	Truth-table verification	PASS	Insert
NOT	NOT.tst	PASS	Insert
AND	And.tst	PASS	Insert
OR	Or.tst	PASS	Insert
Register	Register.tst	PASS	Insert
ALU	ALU.tst	PASS	Insert

Important: Screenshots should clearly show the **module name, test inputs/outputs and PASS status**.

13. Brief Reflection: Why NAND Is

Universal Sample Student Reflection

NAND is called a universal logic gate because all basic logic operations can be constructed using only NAND gates. For example, a NOT gate can be obtained by connecting both inputs of a NAND gate together. An AND gate can be created by connecting a NAND gate followed by another NAND gate used as an inverter. Similarly, an OR gate can be implemented using NAND gates by applying De Morgan's theorem. Since NOT, AND, and OR are sufficient to construct any Boolean logic circuit, NAND gates can theoretically be used to build complete digital systems. This demonstrates that a single type of simple logic gate can serve as the fundamental building block for complex digital hardware.

14. Learning Outcome

After completing this activity, I understood:

1. How basic logic gates can be constructed from NAND.
 2. How HDL describes hardware rather than conventional sequential software.
 3. How individual chips can be combined to construct larger circuits.
 4. How a 16-bit ALU performs different arithmetic and logical functions.
 5. How simulation is used to verify hardware functionality.
 6. Why NAND is considered a universal logic gate.
 7. The importance of systematic testing and debugging in digital circuit design.
-

15. Conclusion

The experiment successfully demonstrated the implementation and verification of digital logic circuits using HDL. Basic gates were constructed using NAND logic, followed by the development and testing of larger components such as registers and the ALU. Simulation results were used to verify the correctness of the HDL implementation. The activity also demonstrated the universality of NAND gates and provided practical understanding of hierarchical digital system design.
